Ders 19 Genişletilmiş Notu: Uygulama: Thread'ler ve Senkronizasyon, Dosya İşlemleri, Tip Dönüşümü ve İç Sınıflar

Ders 19 Genişletilmiş Notu: Uygulama: Thread'ler ve Senkronizasyon, Dosya İşlemleri, Tip Dönüşümü ve İç Sınıflar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Program, Proses ve Thread
- 2. İlk Örnek: Thread Oluşturmak
- 3. Beklenmedik Sonuçlar: Statik Sayaç ve Thread.sleep
- 4. Kitapçı Simülasyonu ve synchronized
- 5. Thread ile Paralel Toplama
- 6. Metin Dosyasına Yazma ve Okuma
- 7. Nesneyi Dosyaya Yazma ve Okuma
- 8. Tip Dönüşümü (Typecasting)
- 9. İç Sınıflar (Inner Class)

Bu oturum Zoom üzerinden yapılan bir uygulama dersidir ve dersin asistanı tarafından yürütülür. Öğrenciler ona "hocam" diye seslenir, bu notta "asistan" diye geçer. Asistan oturumun thread ağırlıklı olacağını, diğer konulardan da örnekler göstereceğini baştan söyledi. Sıra şöyle ilerledi: önce thread'ler, sonra dosya işlemleri, en sonda da daha kısa geçilen tip dönüşümü ve iç sınıflar.

Bu nottaki kodlar slaytın ya da ekranın aynısı değildir. Altyazıda kodlar yalnız sözlü anlatılıyor, bu yüzden her biri anlatımdan kuruldu, sınıf ve değişken adlarının bir kısmı buna göre verildi. Tam programların hepsi JDK 25 ile derlenip çalıştırıldı.

1. Program, Proses ve Thread

Asistana göre thread'i anlamanın en iyi yolu onu üç kavramlı bir dizinin içinde görmektir: program, proses ve thread.

- **Program:** Masaüstünde duran, `.exe` uzantılı, çalıştırabildiğiniz şeydir.
- **Proses (süreç):** Programın çalışan hâlidir. Çalıştırılmıyorken program olan şey, "run"a basıldığında prosese dönüşür. Asistanın örneği Zoom'du: Zoom uygulamasını açmadan önce o bir programdı, açıldıktan sonra bir prosestir.
- **Thread (iş parçacığı):** Her prosesin bir thread'i olur, birden fazla da olabilir.

Özel olarak bir thread oluşturmadıysanız, şimdiye kadar yazdığınız bütün programlar tek thread'lidir (single thread). Bu derste birden fazla thread oluşturmayı göreceğiz.

Birden fazla thread olması, programda birden fazla akış olması demektir. Normal bir programda önce bir satır çalışır, sonra alttaki, sonra onun altındaki. Her satır üstündekinin bitmesini bekler. Birden fazla thread olduğunda ise bir thread bir satırı çalıştırırken başka bir thread başka bir satırı çalıştırabilir. Modern işlemciler çok çekirdekli (multi core) olduğu için bu akışlar gerçekten eş zamanlı çalışabilir.

2. İlk Örnek: Thread Oluşturmak

Asistan, thread'lerin aynı anda farklı akışlardan gittiğini göstermek için çok basit bir yazdırma işlemi seçti. Aynı işi önce thread'siz, sonra thread'li yaptı.

2.1 Thread'siz Hâl: SimpleClass

`SimpleClass` sıradan bir sınıftır. Yapıcısında bir isim alır, `run` adında normal bir metodu vardır. Metot önce başladığını yazar, sonra aynı isimle 5'e kadar sayıları yazar, en sonda bittiğini yazar.

Bu sınıftan `sc1` ve `sc2` adlı iki nesne oluşturulur ve metotları sırayla çağrılır. Beklenen de görülen de aynıdır: önce birincisi başlar ve biter, ondan sonra ikincisi başlar ve biter. Çıktının sırası her çalıştırmada aynıdır.

2.2 extends Thread ile Thread Oluşturmak

Java'da thread oluşturmanın iki yolu vardır:

- `extends` ile, yani kalıtımla.
- `implements` ile, yani arayüzle.

Asistan önce daha basit olan kalıtım yolunu gösterdi. `SimpleThread` sınıfı `extends Thread` der. Java'da bir thread çalışıyorsa `run` metodunun içinde çalışır, bu yüzden yapılacak iş `run` içine yazılır. Burada `run` parametre almaz. İsmi yine çıktıda ayırt edebilmek için yapıcıdan veriyoruz. Yaptığı iş `SimpleClass` ile birebir aynıdır.

2.3 start ve run Farkı

Nesneler oluşturulduktan sonra thread'ler `start` ile başlatılır. Burada bir incelik vardır: çalışacak metot `run` olduğu hâlde biz `start` çağırırız, çünkü thread'i başlatan `start`'tır.

```
t1.start();
t2.start();
```

> **Hocanın vurgusu:** `t1.run()` yazılsaydı kod normal bir program gibi çalışırdı. Thread olarak çalışması için `start` çağrılmalıdır.

Bu söz denendi: `run` doğrudan çağrılınca kod `main` akışında çalıştı, `start` çağrılınca ayrı bir thread'de çalıştı (javac ile denendi).

2.4 Çalışma Sırası Belirsizdir

Thread'li program çalıştırılınca ilk örnekten farklı bir şey olur. Birinci thread başladığını yazar, sonra ikinci başladığını yazar. İkinci biraz devam eder, araya birinci girer, sonra ikinci yine devam eder. Aynı kod bir daha çalıştırıldığında sıra yine değişir: bu sefer birinci 5'e kadar gider, ondan sonra ikinci çalışır.

> **Hocanın vurgusu:** Burada deterministik bir durum yoktur. Thread'ler o anda işletim sisteminin ayırdığı kaynaklara göre çalışır.

2.5 join: Thread'lerin Bitmesini Beklemek

Asistan `main`'in sonuna "Normalde son satır olması lazım" yazdıran bir satır ekleyip bunun nerede görüneceğini sordu. Program çalışınca bu yazı en başta çıktı.

Sebebi şudur: `start` dendiğinde bir thread kendi akışında, diğer thread kendi akışında ilerlemeye başlar. Programın ana akışı ise kaldığı yerden aynen devam eder. Thread'ler işe başlayıp bir şey yazdırana kadar ana akış çoktan bir sonraki satıra gelmiştir.

Thread'lerin işi bitmeden devam etmek istemiyorsak `join` kullanırız:

```
t1.join();
t2.join();
System.out.println("Normalde son satır olması lazım");
```

İki thread için de `join` dendiğinde, ana akış ikisinin de bitmesini bekler ve yazı gerçekten son satırda görünür.

2.6 Öncelik: setPriority

Thread'lere öncelik de verilebilir. Thread oluşturulduktan sonra `setPriority` çağrılır. Sayı ne kadar yüksekse öncelik o kadar yüksektir. Asistan bunun bir garanti olmadığını özellikle söyledi. Örnekteki iş çok kısa olduğu için önceliği yüksek olan thread'in daha önce bittiği görülmez.

> **Ek bilgi (derste anlatılmadı):** `setPriority` 1 ile 10 arasında değer alır. `Thread.MIN_PRIORITY` 1, `Thread.NORM_PRIORITY` 5, `Thread.MAX_PRIORITY` 10'dur ve 11 verilince `IllegalArgumentException` oluşur (javac ile denendi). Bölüm 3.2'de geçen `Thread.sleep` ise `InterruptedException` fırlatabildiği için `try-catch` içinde ya da `throws` ile yazılmak zorundadır, yoksa kod derlenmez (javac ile denendi).

2.7 implements Runnable ile Thread Oluşturmak

`extends Thread` thread oluşturmanın bir yoludur, ama asistan "aslında biz thread'leri böyle oluşturmayız" dedi. İkisi de çalışır, fakat arayüzle oluşturmak daha doğru bir kullanımdır.

> **Hocanın vurgusu:** Java'da tek bir `extends` hakkımız vardır. Bunu `Thread` için harcamak istemeyiz, tasarım kararlarında kullanmak isteriz. `implements` ise istediğimiz kadar yapılabilir.

`implements Runnable` yazıldığı anda geliştirme ortamı uyarı verir: bütün soyut metotlar uygulanmalıdır. Yani `run` metodu yazılmak zorundadır. Bu metot thread oluştuğunda çalışacak olan koddur.

Asistan bunu önceki uygulamadaki örneğe benzetti. Orada bir `Comparable` arayüzü tanımlamış, içine "büyüktür mü" metodunu koymuştuk. Arayüzü uygulayan sınıf bu metodu yazmak zorunda kalıyordu. Burada da `Runnable` arayüzünü biri aynı şekilde yazmış, içine `run` koymuş, biz de onu uyguluyoruz.

Bu yolda önemli bir fark vardır: `extends Thread` ile yazılan sınıf doğrudan bir thread'dir, `Runnable` uygulayan sınıf ise doğrudan thread olmaz. Thread oluştururken `new Thread` yazıp nesnemizi bunun içine veririz:

```
Thread r1 = new Thread(new SimpleRunnable("Runnable 1"));
r1.start();
```

Burada iki tane `new`, yani iki nesne vardır: arayüzü uygulayan sınıfın nesnesi ve onu içine alan `Thread` nesnesi. `extends` yolunda ise tek nesne vardır. Sonrası aynıdır: `start` ve istenirse `join`. Çalıştırınca yine farklı bir sıra görülür. Asistanın denemesinde birinci önce başlamış, ama ikinci önce bitmişti.

Aşağıdaki program bu bölümün üç hâlini bir araya getirir (javac ile denendi):

```
class SimpleClass {
    private String name;
    SimpleClass(String name) { this.name = name; }
    public void run() {
        System.out.println(name + " başladı");
        for (int i = 1; i <= 5; i++) System.out.println(name + " " + i);
        System.out.println(name + " bitti");
    }
}

class SimpleThread extends Thread {
    private String name;
    SimpleThread(String name) { this.name = name; }
    @Override
    public void run() {
        System.out.println(name + " başladı");
```

```
        for (int i = 1; i <= 5; i++) System.out.println(name + " " + i);
        System.out.println(name + " bitti");
    }
}

class SimpleRunnable implements Runnable {
    private String name;
    SimpleRunnable(String name) { this.name = name; }
    @Override
    public void run() {
        System.out.println(name + " başladı");
        for (int i = 1; i <= 5; i++) System.out.println(name + " " + i);
        System.out.println(name + " bitti");
    }
}

public class ThreadGiris {
    public static void main(String[] args) throws InterruptedException {
        // 1. Thread yok: sırayla çalışır
        SimpleClass sc1 = new SimpleClass("Sınıf 1");
        SimpleClass sc2 = new SimpleClass("Sınıf 2");
        sc1.run();
        sc2.run();

        // 2. extends Thread
        SimpleThread t1 = new SimpleThread("Thread 1");
        SimpleThread t2 = new SimpleThread("Thread 2");
        t1.setPriority(1);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Normalde son satır olması lazım");

        // 3. implements Runnable
        Thread r1 = new Thread(new SimpleRunnable("Runnable 1"));
        Thread r2 = new Thread(new SimpleRunnable("Runnable 2"));
        r1.start();
        r2.start();
        r1.join();
        r2.join();
    }
}
```

3. Beklenmedik Sonuçlar: Statik Sayaç ve Thread.sleep

Thread'lere girişten sonra asistan, beklenmedik durumların da olabildiğini göstermeye geçti.

3.1 İkinci Örnek: Statik Sayaç

Asistan önce statik değişkenleri hatırlattı. Statik değişken arttırıldığında değer her seferinde baştaki 5'e dönmez, 6, 7, 8 diye kaldığı yerden devam eder. Bu örnekte de 5'ten başlayan statik bir sayı var. Sınıfın `add` metodu sayıyı bir arttırır, sonra sayının değerini yazar. Thread'de çalışan metot `run` olduğu için `add`, `run`'ın içinden çağrılır. İki thread bu metodu birer kez çalıştırır.

Aynı kod art arda çalıştırıldığında asistanın ekranındaki sonuçlar şöyleydi:

Çalıştırma	Çıktı
1	7, 7
2	7, 7
3	6, 7

Aynı kod bazen 7-7, bazen 6-7 yazıyor. Bunu anlamak için iki olası sırayı adım adım izleyelim. Başlangıçta sayı 5'tir.

Birinci sıra (çıktı 6, 7):

1. Birinci thread sayıyı arttırır: 5'ten 6'ya.
2. Birinci thread "değer 6" yazar.
3. İkinci thread sayıyı arttırır: 6'dan 7'ye.
4. İkinci thread "değer 7" yazar.

İkinci sıra (çıktı 7, 7):

1. Birinci thread sayıyı arttırır: 5'ten 6'ya.
2. Birinci thread yazdırma satırına geçmeden ikinci thread gelir, sayıyı 6'dan 7'ye arttırır.
3. Birinci thread yazdırma satırına geçer. O anda sayı 7 olduğu için 7 yazar.
4. İkinci thread de yazdırma satırına geçer ve 7 yazar.

> **Ders sonrası not:** Asistan iki sıra gösterdi, ama üçüncü bir sonuç da mümkündür. Aşağıdaki kod 300 kez çalıştırıldığında 139 kez 6-7, 160 kez 7-7 ve 1 kez 6-6 çıktı (javac ile denendi). Çıktı 6-6 ise iki arttırmadan biri kaybolmuş demektir, sebebi Bölüm 4.2'deki atomik olmama durumudur.

```
class Sayac {
    static int sayi = 5;
    static void add(String ad) {
        sayi++;
        System.out.println(ad + ": sayının değeri " + sayi);
    }
}

class SayacThread extends Thread {
    private String ad;
    SayacThread(String ad) { this.ad = ad; }
    @Override
    public void run() { Sayac.add(ad); }
}

public class StatikSayac {
    public static void main(String[] args) throws InterruptedException {
        SayacThread t1 = new SayacThread("Thread 1");
        SayacThread t2 = new SayacThread("Thread 2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();
    }
}
```

3.2 Üçüncü Örnek: Thread.sleep

Asistan sonraki örneklerde kullanacağı için `sleep`'i ayrı bir küçük örnekle gösterdi. İçine milisaniye cinsinden süre verilir. 4000 yazılınca 4000 / 1000 = 4 saniye boyunca program hiçbir şey yapmaz, uyur, sonra yazdırmaya devam eder.

```
public class Uyku {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("Uyumadan önce");
```

```
        Thread.sleep(4000);
        System.out.println("4 saniye sonra");
    }
}
```

4. Kitapçı Simülasyonu ve synchronized

Asistan, thread'lerin işe yaradığı bir örneğe geçmeden önce sorun yaratma ihtimali olan bir örnek gösterdi.

4.1 Dördüncü Örnek: Kitapçı

`Kitapci` sınıfının iki metodu vardır:

- **`kitapAl(fiyat)`:** Kitap sayısı artar, bakiye fiyat kadar azalır.
- **`kitapSat(fiyat)`:** Kitap sayısı azalır, bakiye fiyat kadar artar.

Kurgu şöyledir: Bir thread 10 kitap alır, başka bir thread 10 kitap satar, ikisi de aynı kitapçı üzerinde çalışır. Bir thread'de tek bir `run` metodu çalıştığı için iki ayrı sınıf yazılır. Biri `run` içinde alır, öteki `run` içinde satar. İkisi de aynı kitapçıyı yapıcıdan alır, böylece aynı nesne üzerinde işlem yapar.

Başlangıç değerleri:

- Kitapçı: bakiye 100 TL, kitap sayısı 100. Bu nesne programın ilk satırında oluşturulur.
- Alış fiyatı 10 TL, satış fiyatı 10 TL. Asistanın deyişiyle kâr amacı yok.
- Alım satım sayısı 10.
- Gecikme: Her alım ya da satımdan sonra `Thread.sleep` ile beklenecek süre.
- Simülasyon sayısı: Aynı işlemin kaç kez tekrarlanacağı.

Alıcı ve satıcı thread'ler yapıcılarında fiyatı, kitapçıyı, adedi ve gecikmeyi alır. `run` metodu adet kadar döner, her turda bir işlem yapıp gecikme kadar uyur. Döngünün her turunda birer alıcı ve satıcı thread'i oluşturulur, `start` ile başlatılır. Döngünün başına dönmeden bunların bitmesi istendiği için `join` çağrılır.

Beklenen sonucu hesaplayalım:

- Bakiye: 100 - 10 × 10 + 10 × 10 = 100 TL.
- Kitap: 100 + 10 - 10 = 100.

Yani 10 kitap alınıp aynı fiyata 10 kitap satılınca hiçbir şey değişmemelidir (python ile hesaplandı).

Asistan bu kodu parametrelerle şöyle denedi:

Deneme	Simülasyon	Gecikme	Son bakiye	Son kitap	Yorum
1	1	1 ms	100	100	Beklenen
2	1	10 ms	100	100	Beklenen
3	1	10 ms	120	100	Bakiye bozuldu
4	100	10 ms	"çok saçma sayılar"	"çok saçma sayılar"	Hatalar birikti
5, `synchronized` ile	100	10 ms	100	100	Düzeldi

Üçüncü denemede aynı kod, aynı parametrelerle çalıştığı hâlde bakiye durduk yere 120 oldu, kitap sayısı ise 100'de kaldı. Fark 120 - 100 = 20 TL'dir. Her işlem bakiyeyi 10 TL değiştirdiği için bu fark 20 / 10 = 2 işlem etkisi demektir (ara adım eklendi). Asistan dördüncü denemede simülasyon sayısını, hatalar arka arkaya birikip iyice görünsün diye 100 yaptı. Sayılar artık anlamsızdı ve her çalıştırmada rastgele değişiyordu.

4.2 Neden Bozuluyor: Atomik Olmama

Sebep şudur: Thread'ler aynı değişkene erişiyorsa ve biri değişkeni arttırırken öbürü araya girerse değer fazla ya da eksik çıkabilir. Çünkü arttırma işlemi atomik değildir, yani tek bir birim zamanda olup bitmez. Alt seviyede, assembly'de birkaç adımda gerçekleşir. Asistan buna atıyorum altı adım dedi. Tam register'larda bu işlem yapılırken öbür thread aynı değişkene erişip bir şey yaparsa sonuç bozulur.

> **Ek bilgi (derste anlatılmadı):** Tek satırlık `sayi++` için derleyicinin ürettiği bytecode'a bakıldığında statik sayaçta `getstatic`, `iconst_1`, `iadd` ve `putstatic` komutları görülür (javac ve javap ile denendi). Yani değer önce okunur, sonra arttırılır, sonra geri yazılır. İki thread aynı değeri okuyup ikisi de aynı sonucu geri yazarsa bir arttırma kaybolur.
>
> **Hocanın vurgusu:** Bir bankacılık ya da muhasebe uygulamasında böyle bir şey kabul edilemez, olmaması gerekir.

4.3 Çözüm: synchronized

Bu sorunun çözümü dilden dile değişir. Java bu durum için özel bir anahtar kelime yapmıştır, başka dillerde bazen çözümü kendiniz kurmanız gerekir. Asistan, Java'daki kolay çözümü gösterdi: `kitapAl` ve `kitapSat` metotlarının başına `synchronized` yazmak. Bu kelime kodda baştan vardı, asistan "biraz spoiler" diyerek önce silmişti. Geri yazılıp çalıştırılınca sonuç 100 ve 100 oldu, saçma değerler görülmedi.

Soru: Bu anahtar kelime ne yapıyor? Sorunu çözüyor, ama nasıl çözüyor?

Cevap: İki metot da aynı değişkene erişiyor. İkisine de `synchronized` yazıldığında, bir thread o işlemi yaparken Java öbür thread'in bu işlemi yapmasına izin vermez, onu oraya sokmaz. Kontrolü eline alır ve "şu an bu değişken değiştiriliyor, sen dur" der. O işlem bitince öbür thread çalışır. Az önce söylenen altı adımın arasına girilmesini doğrudan engelleyen bir anahtar kelimedir. Bunu C'de yazsaydınız kendiniz uğraşırdınız: Başka değişkenler koyar, "thread bu bölümün içindeyse sen bu işlemi yapma" diye kendiniz ayarlardınız.

> **Ders sonrası not:** Asistan `synchronized` için "değişkene erişimi engeller" der gibi anlattı. Doğrusu, kilit değişkende değil nesnededir. Tek nesne üzerinde iki thread `synchronized` metodu çağırınca hiç arttırma kaybolmadı. İki ayrı nesneyle aynı statik değişkeni arttırınca, ya da aynı değişkene giden metotlardan biri `synchronized` değilken, arttırma kayboldu. Statik değişken için metodun `static synchronized` yazılması sorunu giderdi (javac ile denendi).

Asistan bu konuların İşletim Sistemleri dersinin 3-4 haftalık konusu olduğunu, burada gösterilenin "girişin girişi" sayılacağını söyledi. Orada deadlock, mutual exclusion ve starvation gibi kavramlar ayrıntılı işlenir. Merak eden araştırabilir.

Aşağıdaki program kitapçı örneğini kurar. `synchronized` kelimeleriyle iki denemede de son bakiye 100.0, son kitap sayısı 100 çıktı. İki kelime silinince her çalıştırmada başka, bozuk bir sonuç çıktı (javac ile denendi).

```
class Kitapci {
    private double bakiye;
    private int kitapSayisi;

    Kitapci(double bakiye, int kitapSayisi) {
        this.bakiye = bakiye;
        this.kitapSayisi = kitapSayisi;
    }

    public synchronized void kitapAl(double fiyat) {
        kitapSayisi++;
        bakiye -= fiyat;
    }

    public synchronized void kitapSat(double fiyat) {
```

```
        kitapSayisi--;
        bakiye += fiyat;
    }

    public double getBakiye() { return bakiye; }
    public int getKitapSayisi() { return kitapSayisi; }
}

class KitapAlici extends Thread {
    private Kitapci kitapci;
    private double fiyat;
    private int adet;
    private int gecikme;

    KitapAlici(Kitapci kitapci, double fiyat, int adet, int gecikme) {
        this.kitapci = kitapci;
        this.fiyat = fiyat;
        this.adet = adet;
        this.gecikme = gecikme;
    }

    @Override
    public void run() {
        for (int i = 0; i < adet; i++) {
            kitapci.kitapAl(fiyat);
            try {
                Thread.sleep(gecikme);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

class KitapSatici extends Thread {
    private Kitapci kitapci;
    private double fiyat;
    private int adet;
    private int gecikme;

    KitapSatici(Kitapci kitapci, double fiyat, int adet, int gecikme) {
        this.kitapci = kitapci;
        this.fiyat = fiyat;
        this.adet = adet;
        this.gecikme = gecikme;
    }

    @Override
    public void run() {
        for (int i = 0; i < adet; i++) {
            kitapci.kitapSat(fiyat);
            try {
                Thread.sleep(gecikme);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

```java
public class KitapciSimulasyon {
    public static void main(String[] args) throws InterruptedException {
        Kitapci kitapci = new Kitapci(100, 100);
        double fiyat = 10;
        int adet = 10;
        int gecikme = 10;
        int simSayisi = 100;

        for (int s = 0; s < simSayisi; s++) {
            KitapAlici alici = new KitapAlici(kitapci, fiyat, adet, gecikme);
            KitapSatici satici = new KitapSatici(kitapci, fiyat, adet, gecikme);
            alici.start();
            satici.start();
            alici.join();
            satici.join();
        }

        System.out.println("Son bakiye: " + kitapci.getBakiye());
        System.out.println("Son kitap sayısı: " + kitapci.getKitapSayisi());
    }
}
```

5. Thread ile Paralel Toplama

Akla şu soru gelebilir: Bu kadar sorun varsa, iş bu kadar karışıyorsa neden thread kullanalım? Asistanın cevabı: Çünkü bazen işlemleri hızlandırabilir.

5.1 Hangi İşler Paralel Çalışabilir?

Asistanın ilk örneği matris çarpımıydı. Satırlar ile sütunların çarpımı birbirinden bağımsız işlerdir, birbirini beklemek zorunda değildir. Biri çalışırken öbürü de çalışabilir.

> **Ders sonrası not:** Asistan "birinci satırla birinci sütun çarpılır, ikinci satırla ikinci sütun çarpılır" dedi. Doğrusu, sonuç matrisinin (i, j) elemanı i. satır ile j. sütunun çarpımından gelir, yani her satır her sütunla çarpılır: $c_{ij} = \sum_k a_{ik} b_{kj}$ (python ile hesaplandı). Paralellik sözü bu hâliyle de geçerlidir, çünkü bu çarpımlar birbirini beklemez.

Karşı örnek olarak bir sayının önce karesini alıp sonra ikiyle çarpma işini verdi. Burada önce kare alınmalıdır, adımlar birbirini beklemek zorundadır. Yani her iş paralelleşmez, ama bazı işler paralel çalışabilir.

5.2 Beşinci Örnek: Görev ve Plan

Örnek en basit hâliyle kurulmuştur. Belirli bir sayıda `Integer` içeren bir `ArrayList` oluşturulur. Sayılar rastgeledir, `min` ve `max` ile 0 ile 100 arasında üretilir. Görev bütün elemanları toplamaktır, iki yolla:

1. Tek akışla: `for` döngüsü baştan sona gider, `sum += ...` ile toplar.
2. Beş thread ile: Liste beşe bölünür, her thread beşte birini toplar.

Sonra süreler karşılaştırılır.

5.3 Random ve Seed

Asistan önce rastgelelik konusuna değindi. Tohum vermeden üretilen sayıların toplamı her çalıştırmada farklı çıkar. Sayılar rastgele olduğu için bu mantıklıdır. Ama karşılaştırmanın adil olması için iki yöntemin de aynı sayıları toplaması gerekir. Bu yüzden `Random`'a bir sayı verilir. Asistan 120 yazdı, toplam 972 çıktı. Bir daha çalıştırdı, yine 972 çıktı.

Buna tohum (seed) değeri denir. Bu, tam rastgelelik yerine hep aynı rastgelelik üretilebilsin diye verilir. Buraya herhangi bir sayı yazılabilir, önemli olan aynı sayının kullanılmasıdır.

5.4 Tek Akışla Toplama

`SumCalculator` sınıfı `numbers` listesini, `start` ve `end` değerlerini alır. Normalde sıfırdan başlayıp sona gideriz, ama aynı yapı thread'li hâlde de kullanılsın diye başlangıç ve bitiş parametre yapılmıştır. Thread'siz kullanımda bunlar sıfır ile dizinin sonudur. Metot, o aralıktaki indisleri tek tek alıp toplar, sonucu `sum` değişkenine atar ve döndürür.

Süre şöyle ölçülür:

1. Metot çağrılmadan hemen önce milisaniye cinsinden başlangıç zamanı alınır.
2. Toplama metodu çağrılır.
3. Metot döndükten sonra bitiş zamanı alınır.
4. Bitiş zamanından başlangıç zamanı çıkarılır. Sonuç metodun ne kadar sürdüğüdür.

Asistan farkın görülebilmesi için liste boyutunu arttırdı, çünkü küçük sayılarda fark görünmez. Birkaç kez çalıştırdı, çünkü süre her seferinde farklı çıkar ve hangi aralıkta değiştiğini görmek ister. Ölçümler 152, 149 ve 153 ms oldu.

Asistan programın kendisinin daha uzun sürdüğünü de söyledi. Uzun süren kısım `numbers` listesinin oluşturulmasıdır, çünkü rastgele sayılar tek tek üretilir. Bunu görev olarak düşünmemek gerekir: `numbers` listesinin başka bir yerden geldiğini varsayıyoruz.

5.5 Beş Thread ile Toplama

Thread sınıfı yukarıdakinin aynısı gibi yazılmıştır. `numbers`, `start` ve `end` aynıdır. Fark şudur: `run` bir değer döndüremediği için toplam `run` içinden çağrılan metotta hesaplanıp sınıfın değişkenine yazılır. Thread bittikten sonra bu değer `getSum` ile alınır.

Liste beş parçaya bölünür. Parça uzunluğu liste boyutunun beşte biridir ve 1, 2, 3, 4 ile çarpılarak sınırlar bulunur:

- Birinci thread: 0'dan `part`'a kadar.
- İkinci thread: `part`'tan `2 * part`'a kadar.
- Üçüncü thread: `2 * part`'tan `3 * part`'a kadar.
- Dördüncü thread: `3 * part`'tan `4 * part`'a kadar.
- Beşinci thread: `4 * part`'tan listenin sonuna kadar.

Başlangıç zamanı thread'ler oluşturulmadan önce, bitiş zamanı `join`'lerden sonra alınır. Her thread yalnız kendi beşte birini topladığı için en sonda beş kısmi toplam da toplanıp tek sonuç elde edilir. Ölçümler 86, 92 ve 92 ms oldu. İki yöntemin ekrandaki toplamları da aynıydı, yani aynı sayılar toplanıp aynı iş yapılıyordu.

5.6 Sonuçların Karşılaştırılması

Ölçümler (ortalamalar python ile hesaplandı, ara adım eklendi):

Yöntem	Ölçümler (ms)	Ortalama (ms)	Asistanın yuvarlaması (ms)
Tek akış	152, 149, 153	151,3	150
Beş thread	86, 92, 92	90	90

Asistanın yuvarlanmış sayılarıyla oran 90 / 150 = 0,6'dır. Yani thread'li süre tek akışın yarısından fazladır ama belirgin biçimde kısadır. Hızlanma 150 / 90 ≈ 1,67 katıdır. Beş thread kullanıldı, ama süre 150 / 5 = 30 ms'ye inmedi. Neden beşte bire düşmedi?

Asistanın saydığı sebepler:

- **Thread oluşturmanın maliyeti:** Thread'leri oluşturmak, başlatmak ve `join` çağırmak ek maliyettir. Fazladan nesneler de tutulur.
- **En yavaş thread:** `join` hepsini bekler. Bir thread çok hızlı bitse bile sonuçların toplanması için hepsinin bitmesi gerekir. Yani en yavaş thread kadar paralelleştirebiliriz.

  Ek bilgi (derste anlatılmadı): Aşağıdaki kodu kendi makinenizde çalıştırırsanız 150 ve 90 ms'yi görmeyi beklemeyin, çünkü sonuç makineye, liste boyutuna ve

JVM'in o anki durumuna bağlıdır. Bu kod iki kez denendiğinde iki toplam aynı çıktı, ama iki çalıştırmada da beş thread'li ölçüm tek akıştan daha uzun sürdü (javac ile denendi). Yani paralelleştirmenin hız getirmesi garanti değildir, derste görülen kazanç bir ölçümdür.

Asistan paralelliğin neden gerektiğini basit bir örnekle bağladı: Oyunlarda siz yürürken karşınızdaki karakter de yürümelidir. "Ben biraz yürüyeyim, o dursun, sonra o biraz yürüsün" olmaz.

Aşağıdaki program anlatımı derlenebilir hale getirir. `N` değeri yer tutucudur.

```
import java.util.ArrayList;
import java.util.Random;

class SumCalculator {
    private ArrayList<Integer> numbers;
    private int start, end;
    private long sum;

    SumCalculator(ArrayList<Integer> numbers, int start, int end) {
        this.numbers = numbers;
        this.start = start;
        this.end = end;
    }

    public long getSum() {
        sum = 0;
        for (int i = start; i < end; i++) sum += numbers.get(i);
        return sum;
    }
}

class SumThread extends Thread {
    private ArrayList<Integer> numbers;
    private int start, end;
    private long sum;

    SumThread(ArrayList<Integer> numbers, int start, int end) {
        this.numbers = numbers;
        this.start = start;
        this.end = end;
    }

    private void calculate() {
        sum = 0;
        for (int i = start; i < end; i++) sum += numbers.get(i);
    }

    @Override
    public void run() { calculate(); }

    public long getSum() { return sum; }
}

public class ParalelToplam {
    static final int N = 10_000_000;

    public static void main(String[] args) throws InterruptedException {
        int min = 0, max = 100;
        Random random = new Random(120);
        ArrayList<Integer> numbers = new ArrayList<>(N);
```

```
        for (int i = 0; i < N; i++)
            numbers.add(random.nextInt(max - min + 1) + min);

        // Tek akışla toplama
        SumCalculator sc = new SumCalculator(numbers, 0, N);
        long startTime = System.currentTimeMillis();
        long sum = sc.getSum();
        long endTime = System.currentTimeMillis();
        System.out.println("Tek akış toplam: " + sum + ", süre: " + (endTime - startTime) + "

        // Beş thread ile toplama
        int part = N / 5;
        SumThread t1 = new SumThread(numbers, 0, part);
        SumThread t2 = new SumThread(numbers, part, 2 * part);
        SumThread t3 = new SumThread(numbers, 2 * part, 3 * part);
        SumThread t4 = new SumThread(numbers, 3 * part, 4 * part);
        SumThread t5 = new SumThread(numbers, 4 * part, N);

        startTime = System.currentTimeMillis();
        t1.start(); t2.start(); t3.start(); t4.start(); t5.start();
        t1.join(); t2.join(); t3.join(); t4.join(); t5.join();
        endTime = System.currentTimeMillis();

        long toplam = t1.getSum() + t2.getSum() + t3.getSum() + t4.getSum() + t5.getSum();
        System.out.println("Beş thread toplam: " + toplam + ", süre: " + (endTime - startTime)
    }
}
```

Asistan thread kısmını burada bitirdi.

6. Metin Dosyasına Yazma ve Okuma

6.1 Dosyaya Neden İhtiyaç Var?

Şimdiye kadar değerleri değişkenlerde sakladık, ama program bitince bu değerler de gider. Bunların gitmemesini isteyebiliriz. Asistanın saydığı durumlar:

- Devam eden bir işimiz vardır, yarın kaldığımız yerden hesaplamaya devam edeceğizdir.
- Bir yerden bir şey indirmişizdir, onun üzerinde işlem yapacağızdır.
- Hesapladığımız şeyleri kaydetmek isteriz.
- Doğrudan dosyadan okuyup işlem yapmak isteriz.

Her programlama modelinde dosyalarla bir şeyler yapılır.

6.2 Urun Sınıfı

Dosya örneklerinde çok basit bir `Urun` sınıfı kullanılır. Bir ismi, bir de fiyatı vardır. Getter ve setter metotları ile `toString` dışında iş yapan bir metodu yoktur.

```
public class Urun {
    private String isim;
    private double fiyat;

    public Urun(String isim, double fiyat) {
        this.isim = isim;
        this.fiyat = fiyat;
    }

    public String getIsim() { return isim; }
    public void setIsim(String isim) { this.isim = isim; }
    public double getFiyat() { return fiyat; }
    public void setFiyat(double fiyat) { this.fiyat = fiyat; }
```

```
    @Override
    public String toString() { return "Urun{isim=" + isim + ", fiyat=" + fiyat + "}"; }
}
```

6.3 FileWriter ile Yazma

Bir `ArrayList` oluşturulur, içine iki ürün eklenir. Dosyaya her satıra bir ürün gelecek biçimde, önce isim, sonra virgül ve boşluk, sonra fiyat yazılır. Adımlar şöyledir:

1. Bir `FileWriter` nesnesi oluşturulur. Örnekteki adı `myWriter`'dır. Yapıcıya verilen string, oluşturulacak dosyanın adıdır.
2. `myWriter.write(...)` içine verilen string dosyaya yazılır. Her ürün için `getIsim`, virgül ve boşluk, `getFiyat` ve yeni satır karakteri birleştirilir.
3. Bütün ürünler yazılınca `myWriter.close()` ile dosya kapatılır. Asistan dosyayı kapatmak gerektiğini söyledi.
4. Bir hata olursa `catch` bloğuna düşülür ve hata oluştuğu yazılır.

Çalıştırınca "dosya başarıyla yazıldı" mesajı görülür, yani `catch` bloğuna hiç girilmez. Dosya oluşur ve içinde beklenen satırlar vardır.

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;

public class DosyaYaz {
    public static void main(String[] args) {
        ArrayList<Urun> urunler = new ArrayList<>();
        urunler.add(new Urun("Kalem", 10.1));
        urunler.add(new Urun("Defter", 20.2));

        try {
            FileWriter myWriter = new FileWriter("urunler.txt");
            for (Urun urun : urunler) {
                myWriter.write(urun.getIsim() + ", " + urun.getFiyat() + "\n");
            }
            myWriter.close();
            System.out.println("Dosya başarıyla yazıldı.");
        } catch (IOException e) {
            System.out.println("Hata oluştu.");
        }
    }
}
```

6.4 Scanner ile Okuma

Şimdi `urunler.txt` dosyasının bize geldiğini düşünelim. Hedef, içindekileri okumak, her satırdan bir `Urun` nesnesi oluşturmak ve bunları bir `ArrayList`'e eklemektir.

1. `new File(...)` ile dosya adını veren bir `File` nesnesi oluşturulur.
2. Bu nesne bir `Scanner`'a verilir.
3. `while (myReader.hasNextLine())` döngüsü kurulur. Dosya açıldığında okuyucunun nerede olduğunu tutması gerekir. Başta ilk satırın biraz üstünde olduğunu düşünebiliriz. Bir sonraki satır varsa `hasNextLine` `true` döner ve döngü devam eder. Böylece dosya üç satır da olsa bin satır da olsa okunabilir.
4. `nextLine` bütün satırı tek bir `String` olarak verir.
5. `split(", ")` satırı ayraçtan böler ve bir `String` dizisine dönüştürür. Değer iki tane değil, beş tane de olabilirdi. Bu örnekte `tokens[0]` ürünün ismi, `tokens[1]` ise fiyatıdır.
6. Fiyat bir `double` olmalıdır, çünkü sınıftaki alan `double`'dır. Ama okunan her şey `String`'dir ve bir `String` ile çarpma gibi matematiksel işlem yapılamaz. Bu yüzden `Double.parseDouble` kullanılır. Bu metot "10.1" yazısını `double` tipinde 10.1 değerine çevirir.

7. Ürün oluşturulup listeye eklenir. Bu üç satırlık iş tek satırda da yazılabilir. Ekrandaki kodda da tek satırda yazılmıştır.
8. Sonraki satıra geçilir. Satır kalmayınca döngü biter ve dosya kapatılır.

Çalıştırınca ürün nesneleri oluşur ve yazdırılır.

Asistan sonra dosya adını var olmayan bir adla değiştirdi (`urunler2`). Bu kez `catch` bloğuna düşüldü ve dosyanın bulunamadığı yazıldı, çünkü böyle bir dosya yoktur. Program `catch`'ten sonra devam eder ve ürünleri yazdırmaya çalışır. Liste boş olduğu için boş liste yazar (javac ile denendi).

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.ArrayList;
import java.util.Scanner;

public class DosyaOku {
    public static void main(String[] args) {
        ArrayList<Urun> urunler = new ArrayList<>();

        try {
            File dosya = new File("urunler.txt");
            Scanner myReader = new Scanner(dosya);
            while (myReader.hasNextLine()) {
                String data = myReader.nextLine();
                String[] tokens = data.split(", ");
                System.out.println(tokens[0] + "\t" + tokens[1]);
                urunler.add(new Urun(tokens[0], Double.parseDouble(tokens[1])));
            }
            myReader.close();
        } catch (FileNotFoundException e) {
            System.out.println("Dosya bulunamadı.");
        }

        System.out.println(urunler);
    }
}
```

7. Nesneyi Dosyaya Yazma ve Okuma

Az önce dosyaya string yazdık, gerisini biz yaptık. Bir de doğrudan nesnenin kendisini dosyaya yazabiliriz.

7.1 Önce Hata: NotSerializableException

Asistan önce yazamadığımızı göstermek istedi. Az önceki `Urun` sınıfından bir nesne oluşturulur.

1. Dosya adıyla bir `FileOutputStream` oluşturulur.
2. Bu, bir `ObjectOutputStream` içine verilir. Asistan bu iki satırı ezbere bilmediğini, istenirse iç içe tek satırda da yazılabileceğini söyledi.
3. `writeObject(...)` ile nesne yazılır. İsminden belli olduğu gibi nesneyi yazar.
4. Akışlar kapatılır. Hata olursa `catch` içinde ne olduğu yazılır.

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class NesneYaz {
    public static void main(String[] args) {
        try {
            FileOutputStream fileOut = new FileOutputStream("urun.ser");
            ObjectOutputStream out = new ObjectOutputStream(fileOut);
```

```
            out.writeObject(new Urun("Kalem", 10.1));
            out.close();
            fileOut.close();
            System.out.println("Nesne yazıldı.");
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Bu kod çalıştırılınca `java.io.NotSerializableException` görülür (javac ile denendi).

> **Hocanın vurgusu:** Java'da bir nesneyi nesne olarak dosyaya yazabilmek için sınıfın `java.io.Serializable` arayüzünü uygulaması gerekir. `Urun` bunu uygulamadığı için yazılamıyor.

Yazılamasa da dosyaya bir şeyler gelir, ama bu okuyup devam edebileceğimiz bir içerik değildir.

7.2 Serializable ile Yazma

Asistan `Urun`'un birebir aynısı olan, yalnız `implements Serializable` diyen ve ayırt edilsin diye adı değiştirilen bir sınıf yazdı. Bu sınıftan bir nesneyle aynı `writeObject` çağrısı yapılınca nesne başarıyla yazıldı, `catch` bloğuna düşülmedi.

```
import java.io.Serializable;

public class SerializableUrun implements Serializable {
    private String isim;
    private double fiyat;

    public SerializableUrun(String isim, double fiyat) {
        this.isim = isim;
        this.fiyat = fiyat;
    }

    @Override
    public String toString() { return "SerializableUrun{isim=" + isim + ", fiyat=" + fiyat + "
}
```

Dosyaya string yazarken satırları görebiliyor ve anlamlandırabiliyorduk. Burada durum farklıdır. Dosya uzantısı istenildiği gibi verilebilir: `.ser`, `.txt` ya da hiçbiri. Dosya bir metin editöründe açılınca bizim için anlamlı olmayan karakterler görünür.

> **Ders sonrası not:** Asistan bu içerik için "Java ile işletim sistemi kendi arasında anlaşıyor" dedi. Doğrusu, formatı işletim sistemi değil Java'nın nesne akışı belirler. Yazılan dosyanın ilk baytları `ac ed 00 05` idi ve dosyanın içinde sınıf adı ile alan adları okunabiliyordu (javac ile denendi).

Tek nesne değil, örneğin bir bağlı liste de yazılabilir.

7.3 Nesneyi Okuma

Okuma, yazmanın tersidir. `FileInputStream` oluşturulur, bir `ObjectInputStream` içine verilir, `readObject` çağrılır. Asistan okunan değeri doğrudan `Object` tipinde bir değişkene atadı. Bu yapılabilir, çünkü Java'da her şey `Object` sınıfından türemiş gibi düşünülebilir. Yazdırılınca ürün görünür.

İki hata türü yakalanır:

- **`IOException`:** Asistana göre sebep büyük ihtimalle dosyanın olmamasıdır.
- **`ClassNotFoundException`:** Dosyadan okunan sınıf programda yoksa Java nesneyi oluşturamaz, çünkü öyle bir sınıfı bilmez.

Asistan ikincisini de gösterdi. `SerializableUrun` sınıfını kaldırıp okumayı çalıştırınca `ClassNotFoundException` alındı. Yani okunabilmesi için sınıfın bulunması gerekir (javac ile denendi: sınıf dosyası olmadan aynı hata alındı).

```
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class NesneOku {
    public static void main(String[] args) {
        try {
            FileInputStream fileIn = new FileInputStream("urun2.ser");
            ObjectInputStream in = new ObjectInputStream(fileIn);
            Object urun = in.readObject();
            System.out.println(urun);
            in.close();
            fileIn.close();
        } catch (IOException e) {
            System.out.println(e);
        } catch (ClassNotFoundException e) {
            System.out.println(e);
        }
    }
}
```

Bu kodun okuyacağı `urun2.ser`, `NesneYaz`'ın `SerializableUrun` nesnesi yazacak biçimde değiştirilmesiyle oluşturuldu.

Asistan bununla ilgili sürüm numarası yazılabildiğini, bunun ders slaytlarında olduğunu söyledi, örneği basit tutmak için üzerinde durmadı.

> **Ek bilgi (derste anlatılmadı):** Bu sürüm numarası sınıfta `private static final long serialVersionUID` alanıyla verilir. Nesne bir değerle yazılıp sınıftaki değer değiştirildikten sonra okunmaya çalışılınca `java.io.InvalidClassException` oluştu (javac ile denendi).

7.4 Soru-Cevap

Soru: Bir nesneyi dosyaya yazınca birkaç satır alt alta bir şeyler görünüyor. Her nesnenin dosyada ne kadar yer kaplayacağı belli mi? Bir dosyada birçok nesne varsa hepsini birden nasıl okuyacağız?

Cevap: Ne kadar yer kapladığını bilmiyorum. Ama burada tek ürün yazdık. Bir bağlı liste (`LinkedList`) oluşturup içine on ürün eklediğimizi düşünün. Doğrudan o liste dosyaya yazılabilir. Okuyunca on ürün birden okunmuş olur, okuma yine tek satırdır.

Soru: O zaman bir dosyaya birbirinden bağımsız farklı nesneler yazmak mantıklı değil mi?

Cevap: Ben öyle bir kullanım görmedim, bu yüzden bilmiyorum. Birden fazla nesne yazacaksam onları bir veri yapısında tutar, öyle yazarım. Normal `int`'lerde birlikte olan değerleri aynı dizide tutmamız gibi.

> **Ders sonrası not:** Bu iki soru için denemeler yapıldı (javac ile denendi). Aynı sınıfın isim uzunluğu farklı iki nesnesi farklı boyutta dosya üretti, yani yer kaplama nesneye göre değişir. Aynı akışa art arda iki kez `writeObject` çağrılabildi. İki nesne yazıldıkları sırayla iki `readObject` ile okundu, üçüncü okumada `EOFException` oluştu. `LinkedList` ile tek seferde yazıp okuma da çalıştı. Yani asistanın önerdiği veri yapısı yolu çalışır, ama tek yol değildir.

8. Tip Dönüşümü (Typecasting)

Asistan kalan iki konunun çok uzun olmadığını söyledi.

8.1 İlkel Tipler

Bir `int` ve bir `double` değerimiz olsun. Her `double`, `int`'i kapsar, çünkü ondalıklı sayıları da içerir. Bu yüzden `int` değeri doğrudan `double` değişkene atanabilir, sıkıntı olmaz. Bu dönüşüm otomatiktir. Örnekte 911 değeri `double` değişkene atandı ve yazdırılınca 911.0 göründü.

Tersi biraz sıkıntılıdır. 5.9 gibi bir `double`, `int` değişkene atanırken veri kaybolur. Bu kayıp olduğu için bunu başına `(int)` yazarak özellikle belirtmek gerekir.

`char` ile `int` arasında da dönüşüm yapılabilir, bu ASCII değerleriyle ilgilidir. Asistan bunun üzerinde hızlıca geçti. Daha önce dosya örneğinde görülen `Double.parseDouble` da bir dönüşümdür: `String` değeri `double`'a çevirir. `double`'ı `int`'e çevirmek için ise `(int)` kullanılır.

> **Hocanın vurgusu:** `(int)` dönüşümü yuvarlama yapmaz, yalnız sayının tam kısmını alır. 5.1 de 5 olur, 5.9 da 5 olur.

8.2 Sınıflar Arasında Dönüşüm

`A` ve `B` adlı iki sınıf vardır. `B`, `A`'dan kalıtım alır. Metotların nereden çağrıldığı anlaşılsın diye `A`'da `printFromA` vardır, "Inside A" diye yazar. `B`'de `printFromB` vardır ve `B` ayrıca `printFromA` metodunu override eder.

Nesneleri oluşturmanın olası yolları şunlardır:

Satır	Derleme	Çalışma	Açıklama
`A a2 = new A();`	Geçerli	Geçerli	Olağan kullanım
`B b1 = new B();`	Geçerli	Geçerli	Olağan kullanım
`A a1 = new B();`	Geçerli	Geçerli	`A` daha genel olduğu için yapılabilir
`B b = new A();`	Hata		`A` nesnesi doğrudan `B` diye oluşturulamaz
`B b = (B) new A();`	Geçerli	Hata	Derleyici kabul eder, çalışırken `ClassCastException`

Bu beş satırın sonuçları javac ile denendi.

İlginç olan `a1`'dir. Asistan onu "A objesi ama B'den oluşturulmuş" diye anlattı. `a1` üzerinden `printFromB` çağrılamaz:

```
A a1 = new B();
// a1.printFromB();      // derlenmez
((B) a1).printFromB();   // çalışır
```

Ama nesne `B`'den oluşturulduğu için bu aslında yapılabilmelidir. Bunu yapmak için `a1` `B`'ye dönüştürülür (cast) ve dönüştürülmüş hâli bir nesne gibi düşünülüp metot onun üzerinden çağrılır. Böylece üstte yapılamayan çağrı yapılmış olur. Asistana göre diğer çağrılar beklenen davranıştır, normal yollarla yapılamayıp tip dönüşümüyle yapılabilen asıl ilginç durum budur.

> **Ders sonrası not:** "A objesi" ifadesi tam değildir. `a1`'in referans tipi `A`, nesnenin kendisi `B`'dir. `a1.getClass().getName()` `B` verdi. `a1.printFromA()` da `B`'deki override edilmiş sürümü çalıştırdı. `printFromB` çağrısının derlenmemesi ise derleyicinin referans tipine bakmasındandır (javac ile denendi).

```
class A {
    public void printFromA() { System.out.println("Inside A: printFromA"); }
}

class B extends A {
    @Override
    public void printFromA() { System.out.println("Inside B: printFromA"); }
    public void printFromB() { System.out.println("Inside B: printFromB"); }
}
```

```java
public class TipDonusumu {
    public static void main(String[] args) {
        // İlkel tipler
        int sayi = 911;
        double d = sayi;
        System.out.println(d);          // 911.0

        int a = (int) 5.9;
        int b = (int) 5.1;
        System.out.println(a + " " + b); // 5 5

        // Sınıflar
        A a1 = new B();
        A a2 = new A();
        B b1 = new B();

        a1.printFromA();                // Inside B: printFromA
        a2.printFromA();                // Inside A: printFromA
        b1.printFromA();                // Inside B: printFromA
        b1.printFromB();                // Inside B: printFromB

        // a1.printFromB();             // derlenmez
        ((B) a1).printFromB();          // Inside B: printFromB
    }
}
```

9. İç Sınıflar (Inner Class)

9.1 Yazılışı

Asistan son konuya çok kısa değindi, örneği de internette ilk çıkan sonuçtan aldığını söyledi. İç sınıf demek, bir sınıfın içinde başka bir sınıf tanımlamak demektir: `OuterClass` içinde `InnerClass`. Örnekte önce `OuterClass`'tan bir nesne, ardından bu nesne üzerinden `InnerClass`'ın nesnesi oluşturulur ve iki sınıftaki değerler toplanır. Sonuç 15 çıkar.

```java
class OuterClass {
    int x = 10;

    class InnerClass {
        int y = 5;
    }

    static class StaticInner {
        int z = 7;
    }
}

public class IcSinif {
    public static void main(String[] args) {
        OuterClass outer = new OuterClass();
        OuterClass.InnerClass inner = outer.new InnerClass();
        System.out.println(inner.y + outer.x);   // 15
    }
}
```

İç sınıf `private` olabilir. O zaman dışarıdaki bir sınıftan bu biçimde oluşturulamaz ve çalıştırılmak istenince hata alınır. Denemede bu hata derleme sırasında geldi: `OuterClass.InnerClass has private access in OuterClass` (javac ile denendi). İç sınıf `static` de olabilir, içindeki metot da kullanılabilir. Asistan yazılışların ezberlenmesini beklemedi, projede lazım olunca bakılabileceğini, kendisinin de öyle yapacağını söyledi.

9.2 Kalıtımla Farkı ve Kullanım Amacı

Asıl önemli soru, neden iç sınıf kullanıldığıdır.

> **Hocanın vurgusu:** Kalıtım ve iç sınıf birbirine biraz benzer ve karıştırılır, ama farklıdır. Bu farkı bilmek önemlidir.

- **Kalıtım:** Bir nesneden kalıtım alan başka bir nesne oluşur. Bu nesne, üst nesnenin bütün özelliklerini sağlar ve fazladan özellikleri de vardır.
- **İç sınıf:** Dış sınıf olmadan iç sınıfın anlamı yoktur. Böyle bir nesne öteki olmadan var olamaz, var olması ona bağlıdır. O nesne varken vardır.

Asistan bunu ilk uygulamadaki kitaplık ve raf örneğine bağladı. O örnekte kitaplığın içinde raflar vardı. Raflar kitaplığın yapıcısında oluşturuluyordu ve kitaplık silinince raflar da gidiyordu. Ama `Raf` ayrı bir sınıf olduğu için `main`'de de raf oluşturulabiliyordu. Kurgumuz "kitaplık olmadan raf olamaz" diyorsa `Raf`, `Kitaplik`'in içinde bir sınıf olarak yazılabilirdi. Böylece biri gidip `main`'de `Raf` nesnesi oluşturamazdı.

```
class Kitaplik {
    private class Raf { }

    private Raf raf;

    Kitaplik() {
        raf = new Raf();
    }

    boolean rafVar() { return raf != null; }
}

public class KitaplikOrnek {
    public static void main(String[] args) {
        System.out.println(new Kitaplik().rafVar());   // true
    }
}
```

> **Ders sonrası not:** Asistan iç sınıf yazılınca `main`'de raf oluşturulamayacağını, dış sınıf olmadan iç sınıfın anlamı olmadığını söyledi. Bu söz yazılış biçimine göre değişir. `Raf` sade iç sınıfsa `new Kitaplik.Raf()` derlenmez, ama `k.new Raf()` ile bir kitaplık nesnesi üzerinden `main`'de oluşturulabilir. `main`'den tamamen kapatmak için üstteki gibi `private` yazmak gerekir. `static` tanımlanan iç sınıf ise dış nesne olmadan, `new Kitaplik.IcSinifAdi()` biçiminde oluşturuldu (javac ile denendi).

Dersin Toparlaması

Bu uygulamada önce program, proses ve thread kavramları ayrıldı. Thread oluşturmanın iki yolu görüldü, `implements Runnable` yolunun neden tercih edildiği açıklandı. `start`, `join`, `setPriority` ve `Thread.sleep` kullanıldı. Statik sayaç ve kitapçı örnekleriyle, aynı değişkene erişen thread'lerin sonucu bozabildiği ve Java'da `synchronized` ile bunun önlendiği gösterildi. Paralel toplama örneği thread'lerin işi hızlandırabildiğini, ama hızın thread sayısıyla orantılı artmadığını gösterdi. Bu konuların devamı İşletim Sistemleri dersindedir: deadlock, mutual exclusion ve starvation.

İkinci yarıda kalıcı veri için metin dosyasına yazma ve okuma ile `Serializable` nesne yazma ve okuma işlendi. Ardından ilkel tipler ve sınıflar arasında tip dönüşümü, en sonda da iç sınıflar ve iç sınıfın kalıtımdan farkı görüldü.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.